

design

Summary of overall system

In this project I have designed a peer-to-peer file transfer system that utilizes UDP protocol sockets as the base for communication between different nodes, and I have manually implemented flow control utilizing sliding window and selective repeat like logic that is similar to how TCP protocol would do flow control. Each node in the network runs as an individual process of this program that acts as a sender and a receiver, while maintaining the provided `self.server_socket` as the only port we are allowed to receive on, utilizing multiplexing to handle concurrent nodes. Reliability is practiced through many different logics and implementations, some of which includes utilizing sequence numbers, the typical "three-way" handshake of SYN → SYN-ACK → ACK that TCP performs to establish a connection, sliding window / selective-repeat and keeping track of which chunk of data have not received ACK's yet to retransmit in that case, as packet loss is very common in real practice. To support simultaneous file transfer requests (at least 5) at once as per the specification document, I utilized a shared dictionary `self.sessions` where each individual session's packets and address are mapped to their respective, unique transaction ID (generated from the uuid library in Python), that way packets for different processes are organized in a clean manner and no issue such that other processes packets are mixed with other different packets arise. These transaction ID are also used to multiplex any concurrent file transfer request. This system implements the following command that takes input of the filename / path to file name and the requested file will be delivered to the process / node that requested it successfully.

Final Protocol Design

Packet Format

SYN packet

Key	Value
type	SYN
file	name of file
transaction_id	unique generated ID

COUNT packet

Key	Value
type	SYN
transaction_id	unique generated ID
count	total packet count

COUNT-ACK packet

Key	Value
type	COUNT-ACK
transaction_id	unique generated ID

DATA packet

Key	Value
type	DATA
sequence_number	sequence number (represented as counter for each chunk read)
transaction_id	unique generated ID

ACK packet

Key	Value
type	ACK
sequence_number	sequence number (represented as counter for each chunk read)
transaction_id	unique generated ID

FIN packet

Key	Value
type	FIN
transaction_id	unique generated ID

State Transition Diagram

